

7

Multimodality

With the exciting progress of deep learning in different areas such as computer vision, **natural language processing (NLP)**, robotics, and so on, there is an emerging trend to leverage multiple data sources to develop more powerful applications. This is called **multimodality**, which is a way of unifying different information sources, including images, text, speech, and so on.

In this chapter, we will discuss some fundamental progress in multimodality of using deep learning and share with you some advanced novel applications.

What is multimodality learning?

Before we dive deeper, the first question to ask is, what is multimodality/multimodal?

Modality refers to a certain type of information and/or the representation format in which information is stored. For example, humans have various sensory modalities, such as light, sound, and pressure. In our case, we are talking more about how the data is acquired and stored. For example, commonly available modalities include natural language (both spoken or written), visual information (from images or videos), audio (including voice, sounds, and music), **Light Detection and Ranging (LIDAR)** data, depth images, infrared images, functional MRI and physiological signals, **electrocardiogram (ECG)**, and so on.

The path of leveraging multiple data sources for creating real-world applications is more of a need rather than choice, as in reality, dealing with multimodal data is inevitable. For example, when we watch videos, we consume both visual and audio information. When making a decision while driving, we are leveraging visual, audio, touch sensory, and all other information to jointly make the decision. For an application that would perform perfectly for even a simplest human task, information from various data sources is necessary and needs to be jointly processed.

In the past, since feature extraction is mostly hand-crafted, the traditional way of handling multimodality data is to learn models separately on each individual modality and combine the results by voting, using a weighted average, or other probabilistic method. Some very important aspects, for example, joint learning/feature embedding or learning of the association between different modalities, have been missed. This is where deep learning provides great opportunities. As a representation learning approach, deep learning is capable of extracting task-specific features from single-modality or multiple-modality data. The shared representation captures the relationship between different sources, which can help with the following:

- Learning the transformation from one modality to another modality, and the other way around
- Handling missing modalities as long as the relationship between the absent modality has been effectively learned
- Generating joint representation for downstream prediction/decision making

Challenges of multimodality learning

To use the multimodality information, we will face a few core challenges, such as representation, translation, alignment, fusion, and co-learning (non-exclusive). In this section, we will briefly talk about each of them.

Representation

Representation refers to the *computer-interpretable description of the multimodal data* (for example, vector and tensor). It covers the following, but is not limited to:

- How to handle different symbols and signals—for example, in machine translation, Chinese characters and English characters are two distinct linguistic systems; in a self-driving system, point clouds from LIDAR sensors and image pixels from the RGB camera are two distinct sources with distinct characteristics
- How to handle different granularities
- Modality can be either static or sequential
- Different noise distribution
- Unbalanced proportions.

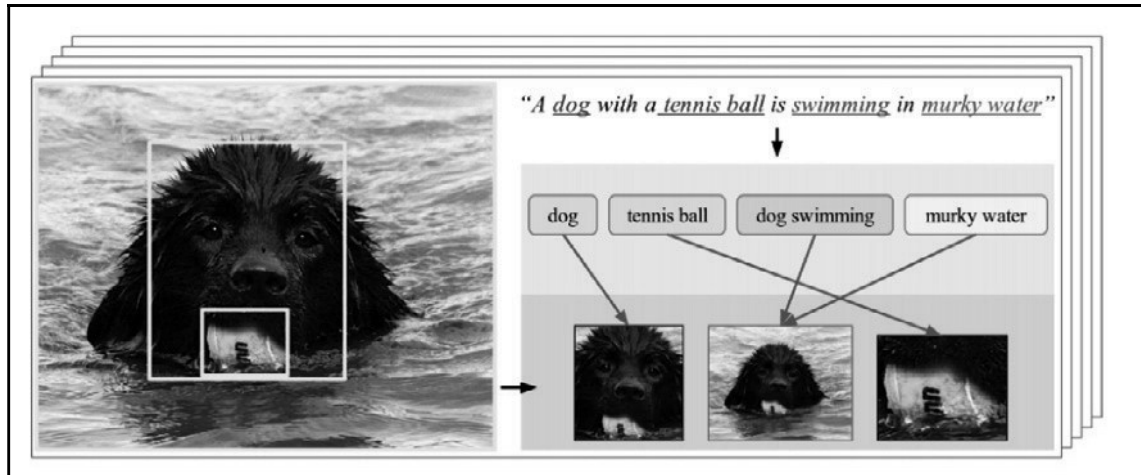
Translation

Translation/mapping refers to *the process of changing data from one modality to another*, for example, image captioning, speech synthesis, visual animations, and so on. Some known challenges include:

- Different representations
- Multiple source modalities
- Open-ended translations
- Subjective evaluation
- Repetitive processes

Alignment

Alignment refers to *identifying relations between elements from two or more different modalities*. The Application examples include image caption alignment, video description alignment, language-gesture co-reference, and lip reading. The following figure shows an example of image caption alignment:



Example of alignment from Andrej Karpathy's paper *Deep Fragment Embeddings for Bidirectional Image Sentence Mapping*

Some known challenges are:

- Long-range dependencies
- Ambiguous segmentation
- Limited annotated datasets with explicit alignments
- One-to-many relationships

Fusion

Fusion refers to *the process of joining information from two or more modalities to perform a prediction*. Example applications include audio-visual speech recognition, multimodal emotion recognition, multimedia event detection, and image-LIDAR joint learning for self-driving. Some known challenges are:

- Different similarity metrics
- Temporal contingency
- Varying predictive power
- Different noise topology
- Ambiguous correspondence

Co-learning

Co-learning refers to *transferring knowledge between modalities and their representations*. Some challenges are as follows:

- Partially-observable views
- Pivot identification
- Collaborative overfitting
- Imperfect predictions
- Potential divergence

In the following section, we will cover a few application cases for some recent benchmarks of multimodality learning. We will be focusing on a high-level introduction of these different fields to give you a better overview and a starting point for how each application can be implemented. With them as an anchor point, you should be able to start your own exploration of deep learning on multimodalities.

Image captioning

With the advanced progress in both computer vision and NLP areas, more and more researchers are starting to look at potential applications in their intersect areas.

One type of application is called **image captioning**, or **im2text**, which is for automatically generating descriptions for a given image. It requires the joint use of technologies in both computer vision and NLP. For a given image, the goal is to analyze its visual content and generate a realistic textual description to describe the major content or most salient aspect of the image. For example, the human in a picture.

To achieve this goal, the caption generation model has to have at least two capabilities:

- Understand the visual cues
- Be able to generate realistic natural language

Understanding the visual cues can be very task-specific; that is, the focus can be different in different scenarios. This is very much like human perception. For example, with the same image but a different focus, the interpretation of the image can be different. It's similar for the text generation step, the same meaning can have a different presentation.

In the following sessions, we will first discuss deep learning techniques, in particular the *show and tell* approach proposed by Oriol Vinyals, to give you a flavor of how deep learning is handling this problem. For this part, we will combine deep learning techniques such as **Convolutional Neural Network (CNN)** (developed in the computer vision area) and **recurrent neural network (RNN)** or **Long Short-Term Memory (LSTM)** (developed in the natural language area) to tackle the problem. It is strongly recommended for the reader to read and understand the content of [Chapter 4, Deep Learning in Computer Vision](#), [Chapter 5, NLP - Vector Representation](#), and [Chapter 6, Advanced Natural Language Processing](#), as they are a preliminary foundation for this chapter.

Following that, we will also briefly discuss some other types of approaches, including traditional methods, such as retrieval-based or template-based approaches for image caption generation.

Certainly, there is more to just generating captions for images at the intersection of these two unrelated fields. Not only can this technique bring us some straightforward applications, such as captioning unlabeled images for better retrieval, tagging, and generating summaries for videos, but it can also potentially help to improve the quality of life for a wide range of the population, for example helping blind people to understand the content captured by a camera or video as a guiding tool. Technologies like this have the potential to make our world more accessible.

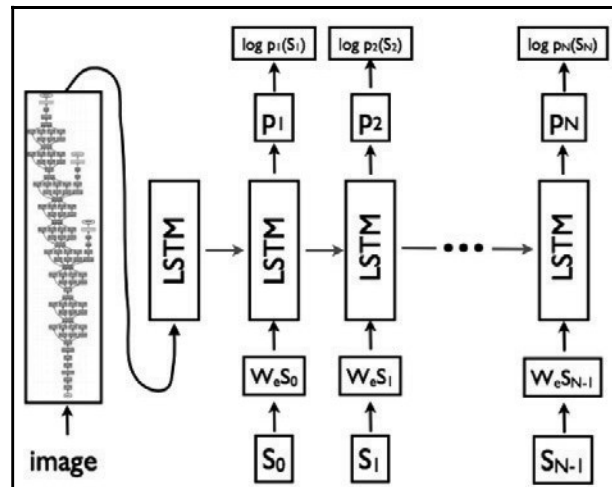
Show and tell

One of the fundamental approaches for image captioning is proposed in Oriol Vinyals' paper *Show and Tell: Lessons learned from the 2015 MSCOCO Image Captioning Challenge* (<https://arxiv.org/abs/1609.06647>). This session provides a deeper dive into the *show and tell* algorithm. At a high-level, the caption model contains two stages: the encoder stage and the decoder stage. The encoder generates an image embedding as the initial input of the decoder, and the decoder transforms the image representation to the natural language description space.

A similar process can be seen in the area of machine translation, where the goal is to translate the content written in one language to another.

Mathematically, this single joint model takes an image, I , as input, and is trained to maximize the conditional likelihood of $p(S|I)$, producing a target sequence of words, $S = \{S_1, S_2, \dots\}$, that can describe the given image accurately/adequately (is the description generated correctly) and fluently (is output grammatical and fluent). Each word, S_t , in this sequence is a one-hot vector representing words from a given dictionary. A similar process can be seen in the area of machine translation, where the goal is to translate the content written in one language to another; that is, given a source sentence, maximize the likelihood of the target sentence $p(T|S)$, where T is the translated sentence, and S is the source sentence. In fact, the approach here is inspired by the machine learning system.

An illustration from Vinyals' paper can be seen as follows:



The preceding illustration shows the LSTM model combined with a CNN image embedder and word embeddings from Oriol Vinyals' paper, *Show and Tell: Lessons learned from the 2015 MSCOCO Image Captioning Challenge*. The unrolling of the LSTM is pictured in blue (dark gray in hard copies). All LSTMs share the same parameters. To understand what *unrolling* is, please refer to Chapter 3, *Getting Started with Neural Networks*.

The following sessions describe each important component in a detailed manner.

Encoder

First, the input image will be encoded by a deep Convolutional Neural Network and generate a vector representation to capture the image information/visual cues in the image.

The CNN can be thought of as an encoder. The last hidden state of the CNN is connected to the decoder, where the state of the neural nodes in this layer is considered as the visual features (for example, with 4096 dimensions). Note that the features are always extracted from the hidden layers of the CNN rather than the last output layer.

Decoder

The decoder, which often uses an RNN or LSTM, will perform language modeling to transform the image representation to sentences. The input to the very first stage of the decoder (at $t = -1$) is the output from the encoder. This is to inform the RNN/LSTM about the image content. During the training stage, we use an embedding transform W_e to embed a word at time t from the ground truth sentence, represented by $W_e S_t$, and feed this embedding to the LSTM at time t to predict the probability distribution of words at time $t+1$, represented by p_{t+1} . The embedding transform W_e can be learned from Word2Vec or other types of embedding techniques.

Training

The output from the last hidden state of the CNN (encoder) is given to the first time step of the decoder. We set $x_1 = \langle \text{START} \rangle$ vector and the desired label y_1 equals the first word in the sequence. Analogously, we set x_2 equals to the word vector of the first word, and expect the network to predict the distribution of second word. Finally, on the last step, x_T equals to the last word, and the target label $y_T = \langle \text{EOS} \rangle$ token. During training, the correct input is given to the decoder at every time step, even if the decoder made a mistake before. Finally, the loss function is defined as the sum of negative log likelihood of the ground truth words:

$$L(I, S) = - \sum_{t=1}^N \log p_t(S_t)$$

Testing/inference

At the inference stage, the image representation is provided to the first time step of the decoder. Set $x_1 = \langle \text{START} \rangle$ vector and compute the distribution over the first word, y_1 . One could either sample a set of words from the distribution, or greedily choose the argmax over the computed probability of the next word, or sample the top ones from the returned candidates (Beam Search). If using LSTM, both the current state and the output will be used in the next time step as input information. The system repeats this process until it hits an $\langle \text{EOS} \rangle$ token.

During testing, the output of the decoder at time t is fed back and becomes the input of the decoder at time $t + 1$ (RNN/LSTM unrolling):



The preceding figure shows a selection of evaluation results, grouped by human rating with four levels—describes without errors, describes with minor errors, somewhat related to the image, and unrelated to the image, from *Show and Tell: Lessons learned from the 2015 MSCOCO Image Captioning Challenge*.

Beam Search

Beam Search is a heuristic search algorithm that explores a graph by expanding the most promising node in a limited set. It can be considered as an optimization of the best-first search that reduces its memory requirements. Basically, it's an iterative approach where, at each step or time t , only consider the set of k best sentences (with length = t) to generate the sentence of $t + 1$. For all resulting sentences with length equals $t + 1$, only keep the best k of them.

Other types of approaches

In general, image captioning approaches can be categorized into two categories—generating the description from scratch, searching, and retrieving from a visual space and or a multimodal space:

- Description as generation from visual input
- Description as a retrieval in visual space or multimodal space

The show and tell model can be considered as generating descriptions from scratch (using an RNN).

For the category of the retrieval types of approaches, there are also two types.

The first type is to treat the problem as a retrieval problem from a joint multimodal space. The training set contains image-description pairs or image patches—sentence segment pairs and a joint model is trained using both visual and textual information. An embedding step is included to build the multimodal space, and a deep learning type of network can be applied in this case. The learnt common representation can be used for cross-modal retrieval, from image to description or from description to image.

The second type is to cast the problem as a retrieval problem in visual space. For example, the system can first explore the similarities in the visual space to collect a set of descriptions. Then, the retrieved top descriptions are used to form the description for the query image.

One problem of the retrieval approach is that it does not aim to generate the novel text. Also, it typically requires a large amount of training data in order to provide a relevant description.

In the past, there have also been approaches that build on top of object detection and recognition, then use a template-based text generation schema or language model to output text. The following figure shows an example of generating tags for image patches through object detection and recognition, then from the generated blobs, construct sentences to generate possible candidates. Candidates are then ranked and the most probable one is chosen as the description. Obviously, one can see that this process did not optimally associate the image and the text. For example, the word **holding** from the final caption is detected and labeled from the women in purple in the image, but is used for describing the woman in brown:

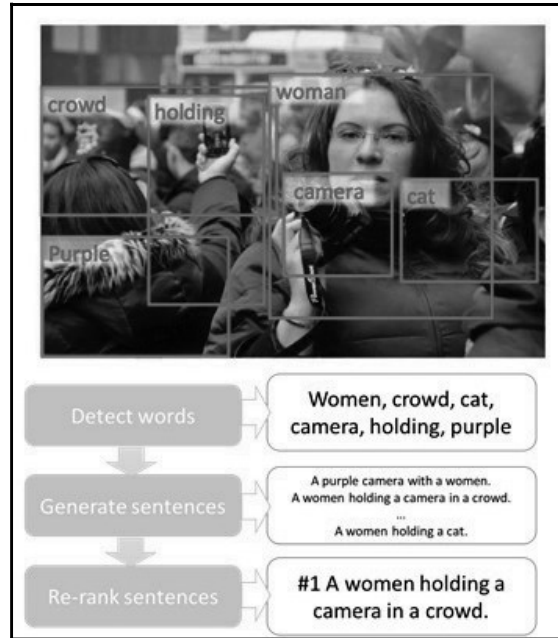


Image captions are generated by detecting and recognizing objects first, then form sentences to select the best possible candidate.

Datasets

There are a few datasets that consist of image and sentence pairs in English describing the content of the images that are available publicly for evaluation purposes:



- Basketball player has fallen on the court while another grabs at the ball from out of frame.
- Two basketball players are scrambling for the ball on the court.
- Two basketball players, one on the floor, struggle to gain possession of a basketball.
- Two high school basketball players reach to grab the ball, one falling to the floor.
- Two men in uniforms playing basketball, struggle for the ball.

The following table lists some basic information:

Dataset name	#Images	#Text	Object	Link	Description	Reference
Pascal VOC 2008	1000	5	No	http://nlp.cs.illinois.edu/HockenmaierGroup/pascal-sentences/index.html	It consists of 1,000 images randomly selected from the training and validation set of the PASCAL 2008 object recognition challenge. Each image is associated with five different captions that describe the entities and events depicted in the image that were collected using crowdsourcing from Amazon Mechanical Turk.	A. Farhadi, M. Hejrati, M. A. Sadeghi, P. Young, C. Rashtchian, J. Hockenmaier, and D. Forsyth, <i>Every picture tells a story: Generating sentences from images</i> , in ECCV, 2010.

Flickr8K	8,092	5	No	http://nlp.cs.illinois.edu/HockenmaierGroup/8k-pictures.html	A collection of 8,000 described images taken from https://www.flickr.com/ . Each image is associated with five different captions that describe the entities and events depicted in the image that were collected using crowdsourcing from Amazon Mechanical Turk. It mainly contains <i>action</i> images of scenes featuring people and animals.	C. Rashtchian, P. Young, M. Hodosh, and J. Hockenmaier, <i>Collecting image annotations using amazon's mechanical turk</i> , in NAACL HLT Workshop on Creating Speech and Language Data with Amazon's Mechanical Turk, 2010, pp. 139–147.
Flickr30K	31,783	5	No	http://shannon.cs.illinois.edu/DenotationGraph/	Contains 31,783 images collected from Flickr. Most of these images depict humans performing various activities. Each image is paired with five crowdsourced captions.	P. Young, A. Lai, M. Hodosh, and J. Hockenmaier, <i>From image descriptions to visual denotations: New similarity metrics for semantic inference over event descriptions</i> , in ACL, 2014.

MSCOCO	164,062	5	Partial	http://mscoco.org/dataset/#overview	The largest image captioning dataset, containing 82,783 images for training, 40,504 images for validation, and 40,775 images for testing. Each image has five human-annotated captions.	T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollar, and C. L. Zitnick, <i>Microsoft COCO: Common Objects in Context</i> , https://arxiv.org/abs/1405.0312 , 2014.
SBU1M	1,000,000	1	No	http://vision.cs.stonybrook.edu/~vicente/sbucaptions/	Also from Flickr but with a totally different distribution. The dataset also comes with pre-computed descriptors. The dataset was initially developed for retrieval-type approaches. The descriptions were originally given by the Flickr user. The author of the data generated the dataset by setting rules on filtering the retrieval results.	V. Ordonez, G. Kulkarni, and T. L. Berg, <i>Im2text: Describing images using 1 million captioned photographs</i> , in NIPS, 2011.

With the exception of SBU, each image has been annotated by labelers with five sentences that are relatively visual and unbiased. SBU consists of descriptions given by image owners when they uploaded them to Flickr. As such, they are not guaranteed to be visual or unbiased and thus this dataset has more noise.

Evaluation

Evaluation of the image caption/description results is not an easy task. Similar to machine translation, some known challenges include, but are not limited to:

- Human evaluation is subjective
- How good is good enough?
- Is system A better than system B?
- Target application and context dependency

Often, two major aspects are considered—linguistic quality and semantic correctness, and two types of approaches are commonly used—human judgments and automatic measures.

The human evaluation is often used through crowdsourcing, for example Amazon Mechanical Turk. The tasks are often designed to include questions on grammar, content, and fluency. Judges are provided with images together with the generated descriptions. In some other cases, the judge might be provided with two images, one with a random description, the other with the model-generated caption, and is asked to select one. This type of approach is good for comparing models, but lack of standardization means that results can be leveraged across different experiments. Also, a human judge may have different agreement levels. Therefore, there is a need for developing an automatic measuring metric for fairness and comparability. Some of the desired attributes of an automated metric include:

- High levels of correlation with quantified human description
- Sensitive to small differences
- Consistent for similar input image/reference pairs
- Reliable—image caption systems that score similarly will perform similarly
- Generalizable—applicable to a wide range of domains and scenarios
- Fast and lightweight, no human involvement

Some of the commonly used automatic evaluation metrics include BLEU [Papineni and their co-authors, 2002, *BLEU: a Method for Automatic Evaluation of Machine Translation*] (precision-based, from the machine translation community), ROUGE [Chin-Yew Lin, *ROUGE: A Package for Automatic Evaluation of Summaries*] (recall-based), METEOR [Denkowski and Lavie, 2014, *METEOR: An Automatic Metric for MT Evaluation with Improved Correlation with Human Judgments*], CIDEr [Vedantam and their co-authors, 2015, *CIDEr : Consensus-based Image Description Evaluation*] and SPICE [Anderson and their co-authors, 2016, *SPICE : Semantic Propositional Image Caption Evaluation*].

BLEU and METEOR were originally developed for evaluating the output of machine translation engines or text summarization systems, while CIDEr and SPICE were designed specifically for image description or captioning evaluation.

All of the approaches are trying to compute the similarity between the algorithm output and one or more human-written/crowdsourced reference texts.

BLEU

Bilingual Evaluation Understudy (BLEU) is a popular metric for machine translation evaluation. It computes an n-gram based precision for the candidate sentence with respect to the references. Intelligibility or grammatical correctness are not taken into account. BLEU computes the geometric mean of the n-gram precisions and adds a brevity penalty (penalizes system results that are shorter than the general length of a reference) to discourage overly short sentences. The range of BLEU is always between zero and one. A number closer to one indicates that the candidate text is more similar to the reference texts. For multiple references, the maximum score is returned as the judgment of quality.

The most common formulation of BLEU is BLEU4, which uses 1-grams up to 4-grams, though lower-order variations such as BLEU1 (unigram BLEU) and BLEU2 (unigram and bigram BLEU) are also used.

For machine translation, BLEU is most often computed at the corpus level, where correlation with human judgment is high (the sentences are first evaluated, but then aggregated to corpus level); for image caption, it is often computed at the sentence level as the accuracies on individual sentences are more of interest.

There are however, some downsides of BLEU. For example, BLEU performs less well when applied at the sentence level or sub-sentence level, and when using only one reference. Part of the reason is because BLEU is n-gram based, and may have many zero or low counts for higher (tri-gram or higher) n-grams. In addition, clipping of n-gram counts, such that they do not exceed the count of each n-gram in the references, also complicates sub-sentential applications.

ROUGE

Recall-Oriented Understudy of Gisting Evaluation (ROUGE). It computes n-gram based recall for the candidate sentence with respect to the references. It is a popular metric for summarization evaluation, trying to answer how often the words (and/or n-grams) in the reference summaries appeared in the machine-generated summaries (compared to BLEU asking how often the words (and/or n-grams) in the machine-generated summaries appeared in the human-reference summaries).

Similar to BLEU, versions of ROUGE can be computed by varying the n-gram count. Two other versions of ROUGE are ROUGE_s and ROUGE_L. ROUGE_s computes an F-measure with a recall bias using skip-bigrams, while ROUGE_L uses the longest common subsequence between the candidate and each reference sentence. Skip-bigrams are a generalization of bigrams in which the words need not be consecutive, but may leave gaps that are skipped over. For multiple references, the maximum score is returned as the judgment of quality.

Note that for computing BLEU and ROUGE, both the generated description and the reference description have to be preprocessed, including tokenization and stripping out non-alphanumeric and hyphen characters. Also, one can apply a stemmer to remove stop words before compute ROUGE scores.

METEOR

Metric for Evaluation of Translation with Explicit ORdering (METEOR) was originally used for measuring machine translation. It combines recall and precision as weighted score components by looking only at the unigram precision, and recalling and aligning output with each reference individually and taking the score of best pairing (instead of BLEU's brevity penalty). It takes into account translation variability via word inflection variations, synonymy and paraphrasing matches, which enables the match between semantic equivalents. Also, it addresses fluency via a direct penalty for word order: how fragmented is the matching of the MT output with the reference? METEOR has significantly better correlation with human judgments compared to BLEU, especially at the segment-level.

CIDeR

Consensus-based Image Description Evaluation (CIDeR) measures the similarity of a generated sentence against a set of ground-truth sentences written by humans, which is trying to solve the problem of weak correlation between the previous metrics and human judgment. CIDeR however shows higher agreement with consensus as assessed by humans. Using sentence similarity, the notions of grammaticality, saliency, importance, and accuracy (precision and recall) are inherently captured by this metric.

SPICE

Semantic Propositional Image Caption Evaluation (SPICE) is based on semantic scene graphs and has shown that it correlates much better with human judgments in comparison to the aforementioned standard measures. The author of this metric illustrated that the previous metrics are sensitive to n-gram overlap, which is neither necessary nor sufficient for two sentences to convey the same meaning. However, SPICE focuses on recovering objects, attributes, and the relations between them; it does not count in the aspects of grammar and syntax. Similar to n-gram based metrics, SPICE implicitly assumes that the caption has been well-formed. It would be recommended to include a fluency metric.

Rank position

For ranking/retrieval-based image generation systems, the ranking position of their original text description can be used for evaluating the system, such as Recall at position k ($R@k$) and median rank scores, where $k = 1, 3, 5$. Recall at position k ($R@k$) means the percentage of test queries for which a model returns the original item among the top k results. Differently, the median rank represents the k at which a system has a recall of 50% (that is the number of results one would have to consider in order to find the original item for half the queries), therefore, the smaller the k is, the better the performance.

Attention models

Attention mechanisms in neural networks are (very) loosely based on the visual attention mechanism found in humans. The idea is to focus on different parts of the inputs as the algorithm tries to make and refine prediction each time.

In computer vision, attention means the ability to attend/focus on a certain region of an image with high resolution while perceiving the surrounding image in low resolution to get better understanding of the salient content, and then adjusting the focal point over time. There are a few benefits of introducing the attention model into image understanding.

First, it helps the model to focus on only the salient or important objects or areas that matter to the viewer of the image (human or computer) and pay less attention or less computation to the background.

Second, it provides the ability to *attend* certain aspects of the image and perform the information extraction/understanding results in a high resolution zoomed-in for that particular region at each time. This ability gives a better chance to understand the local content more precisely. Take an image as an example, such attention helps generate embedding feature vectors that represent local information at a finer-grained level as compared to a global embedding feature vector from the entire image.

The attention model is not a new concept, and has been used in many other applications such as image tracking (Denil and their co-authors, 2011, *Learning where to Attend with Deep Architectures for Image Tracking*).

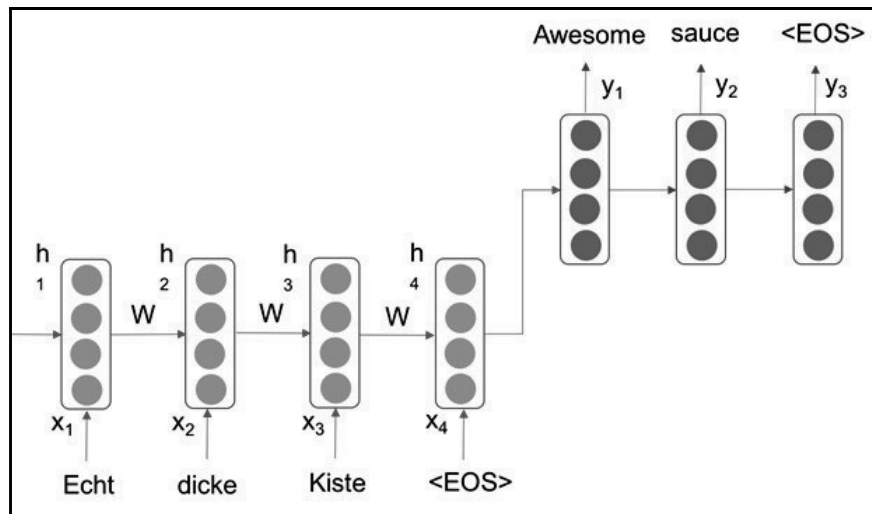
Attention has also been used in natural language processing. We will have a brief introduction to its usage in NLP first in the following session, as this will help us to understand its use in computer vision better.

Attention in NLP

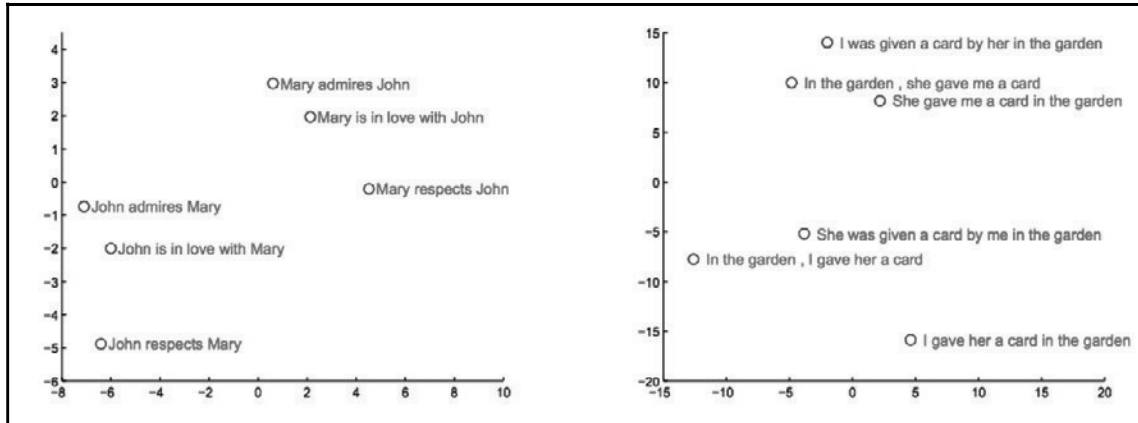
In this session, we will talk about the use of the attention model in RNNs for machine translation.

The problem of machine translation can be formularized as an optimization problem over $P(T|S)$, where S is the source sentence and T is the translated sentence. The machine translation system has two major components: encoder and decoder.

Given an input, S , and each word in this sentence, we can unroll the RNN, so for each time step, the RNN takes an input word together with the previous state to update its internal state. For example, in the following figure, the input words are fed into the RNN's encoder; after the last word, the generated state is essentially the vector representation of the entire sentence. Then, the decoder takes it as an input to generate the probabilities of each translated word in a sequence until it hits an $\langle \text{EOS} \rangle$ mark; that is, basically it is reversing a sentence:



You might wonder how effectively an entire sentence can be represented by a single vector. Indeed, researchers have found that if you plot the sentence embedding to a low dimensional space, sentences with similar meanings would be closer to each other. However, problems do occur when you have longer sentences (we briefly mentioned this in Chapter 5, *NLP - Vector Representation*, when comparing RNNs and LSTM, as LSTM was able to partially solve this problem). The figure below shows that similar sentences are close together in summary-vector space (shown on the left). It also shows the problem with longer sentences:



Two-dimensional visualization of sentence representations from Sutskever and their co-authors' publication *Sequence to Sequence Learning with Neural Networks*

In addition, the problem can get worse if the ways of constructing the sentence in two languages have significant differences in order. For example, in some languages, such as Japanese, the last word might be very important to predict the first word, while translating English to French might be easier as the order of the sentences (how the sentence is organized) is more similar to each other.

One solution is to focus on a part of the sentence, and for each part we generate the prediction of the next word. This sounds like a divide and conquer technique in which we divide the sentences into smaller parts, and assume that in both the source sentence and the target sentence there are local connections between different parts of the sentence.

This saves us the effort of encoding the full-source sentence into a fixed-length vector. So, now the prediction of the word does not singularly depend on the final sentence embedding, or the previous state of the decoder, but on a weighted combination of all states of the encoder, with larger weights on the most correlated parts. The weight of each state reflects how much the prediction of the decoder depends on each input word.

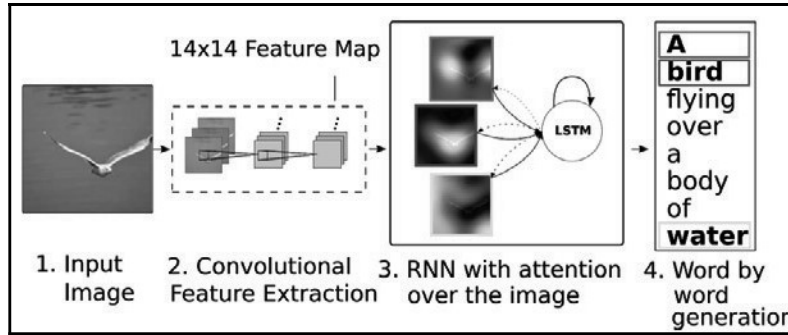
The question now is, how do we generalize or utilize such an idea in computer vision applications?

Attention in computer vision

Similar to the attention mechanism used in machine translation, which helps the neural network to focus on specific parts of the input, such as one to two words at each time step, the attention model also helps the image neural network to focus on different spatial regions or some salient regions for better understanding the image content.

Recall that in the previous session, we discussed how to encode the input image first and use the image embedding as the first time input of the following RNN/LSTM network. Now, the system needs to differentiate different patches/spatial areas of the image as they are not equally important from the perspective of how humans understand the image. Therefore, Xu and their co-authors, in their ground work *Show, attend and tell: Neural Image Caption Generation with Visual Attention*, proposed a way of incorporating an attention mechanism into the system.

Unlike previous work, where feature embeddings are extracted from the fully-connected layers (consider such embeddings are representing the full image, similarly to what we mentioned about the embedding that represents the entire sentence), they first extract the feature from the lower convolutional layer. This allows the decoder to selectively focus on certain parts of an image by selecting a subset of all the feature vectors. The following figure illustrates the architect in the paper of *Show, attend and tell*:



Algorithm architect from Show, Attend and Tell: Neural Image Caption Generation with Visual Attention

As a result, different from the previous model where a single vector/embedding is formed to represent the image information, this time a matrix is formed to represent multiple patches (step two in the preceding figure):

$$a = \{a_1, a_2, \dots, a_L\} \quad a_i \in R^D$$

Where each column is a D-dimensional representation corresponding to a part of the image, and the whole image is represented by a set of L vectors.

Recall that in Chapter 3, *Getting Started with Neural Networks*, where we introduced LSTM, we talked about that at each time step, an LSTM cell has three inputs from the actual input data, the forget gate, and the memory gate.

Suppose the hidden state at $t-1$ can be denoted by h_{t-1} . An attention model, which is also a multilayer perceptron model, outputs the positive weight for each image patch:

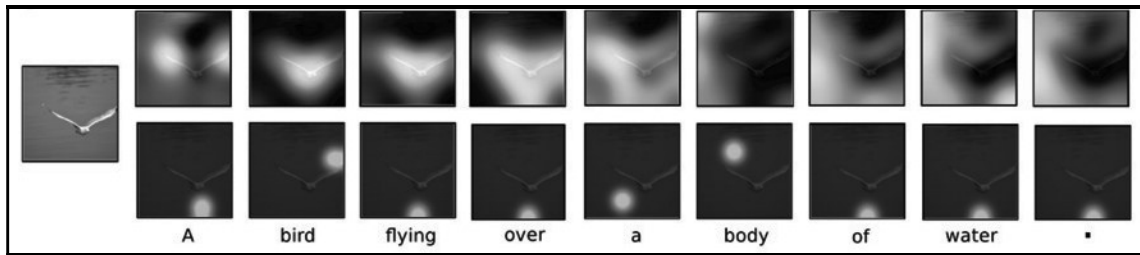
$$e_{ti} = f_{att}(a_i, h_{t-1})$$

$$\alpha_{ti} = \frac{e^{e_{ti}}}{\sum_{k=1}^L e^{e_{tk}}}$$

Where α_{ti} is the weight for patch i at time t . Note that they are typically normalized to sum to one, to form a distribution over the input states. So α_{ti} represents the probability that location/patch i is the right place to focus to produce the next word, or the importance of patch i in all the L patches. With this weight, all the image patch information is dynamically combined together with focus/importance varying each time to generate a context vector, which is used in updating the LSTM cell state and generating the prediction output.

At the training stage, not only have the parameters of the LSTM cell been updated, so has the multilayer perceptron model that determines the weight or importance of each patch.

The following figures are from *Show, Attend and Tell: Neural Image Caption Generation with Visual Attention*, where the first figure illustrates the change of attention over time, and the second figure illustrates attending a particular object in the image and generating the correct word:



The illustration from *Show, Attend and Tell: Neural Image Caption Generation with Visual Attention* shows the attention changes over time as the model generates each word, where the highlighted parts reflect the relevant parts of images regarding the word. The top row is the result of soft attention, and the bottom row is from the hard attention:



The illustration from *Show, Attend and Tell: Neural Image Caption Generation with Visual Attention* shows the correspondence of the attended parts of image (white highlighted part) versus the words that describe that part (with underline).

The difference between hard attention and soft attention

There are two types of attention models: hard attention and soft attention.

In hard attention, each part in a sentence or patch in an image is either used to obtain the context vector or is discarded. In this case, α_i represents the probability of the part/patch being used; that is, the probability of indicator $S_t, i = 1$. For example, in Xu's *Show, Attend and Tell*, the context vector in the hard attention scenario is computed as:

$$z_i = \sum_i s_{t,i} \alpha_i$$

Given the sum-to-one criterion, it can be seen clearly that hard attention selects the element with the highest probability. This is definitely more philosophically appealing, and is also more scalable and efficient regarding computation. However, such a definition is not differentiable. In Xu’s paper, they use Monte Carlo methods to approximate derivatives. Readers can look into the Reinforcement Learning literature for more details.

On the other hand, the soft attention does not have such restriction. Therefore, soft attention is differentiable and one can simply back-propagate the errors to learn the model by gradient descent.

Visual question answering

The task of **visual question answering (VQA)** is the task of answering an open-ended text question about a given image. VQA was proposed by Antol and its co-authors in 2015 (https://www.cv-foundation.org/openaccess/content_iccv_2015/papers/Antol_VQA_Visual_Question_ICCV_2015_paper.pdf). This task lies at the intersection of computer vision and natural language processing. It requires the understanding of the image and the parsing and understanding of the text question. Due to its multimodality nature and its well-defined quantitative evaluation metric, VQA is considered an important artificial intelligence task. It also has potential practical applications, including helping the visually impaired.

A few examples of the VQA task are illustrated in the following table:

		
Q: How many giraffes can be seen? A: 2	Q: Is the bus door open? A: Yes	Q: If you were to encounter this sign, what would you do? A: Stop

Several datasets have been proposed for visual question answering, including, but not limited to, the VQA v1 (http://visualqa.org/vqa_v1_download.html), VQA v2 (<http://visualqa.org/download.html>), visual genome dataset (<http://visualgenome.org/>), and freestyle multilingual *Image Question and Answering* (FM-IQA) dataset (<http://idl.baidu.com/FM-IQA.html>).

In this chapter, we mainly focus on the VQA v1 and v2 datasets. The questions in the VQA dataset are mostly about specifics of the images, so one to three words are often enough for an answer. Let's use the preceding figure as an example. The question is, if you were to encounter this sign, what would you do? We can answer with one word, **Stop**. There are mainly three types of answers: yes/no, number, and other open-ended answers. Experimental results show that machine learning systems get the highest accuracy on yes/no answers, followed by number answers, and other answers. There are also answers that can be obtained from common sense without looking at the image, for example, what is the color of the fire hydrant?

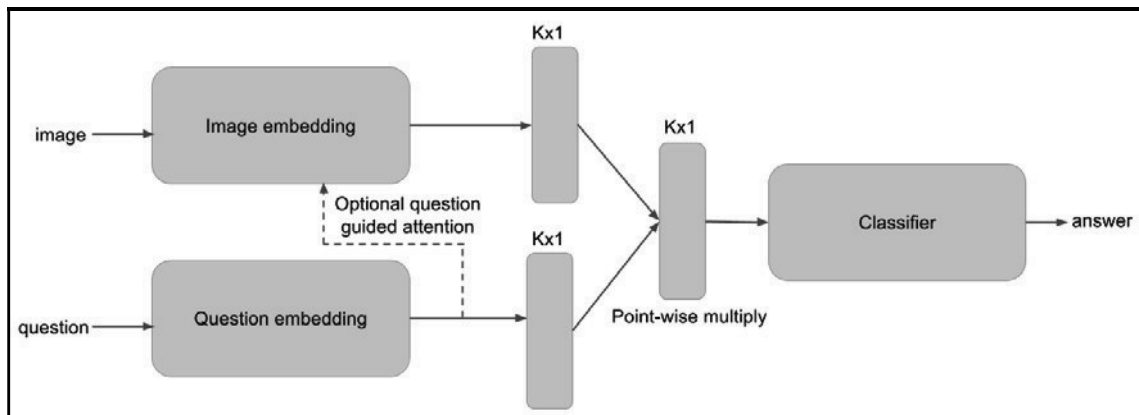
In addition, good performance can be obtained by relying on language-based priors. In other words, just by looking at the questions and training question/answer pairs, one can infer the answer without looking at the image. So VQA v2 uses balanced pairs to emphasize the role of computer vision. It provides two images for one question, and each image leads to a different answer. This discourages blind guesses from questions alone or heavy language-based priors. The VQA v2 dataset consists of 204,721 COCO images, 1,105,904 questions, and 11,059,040 answers provided by human annotators.

The evaluation metric for the VQA dataset (with 10 human annotators) is that, for a particular answer produced by AI, the accuracy of it is:

$$Accuracy(AI_{answer}) = \min\left(\frac{\text{number of humans said } AI_{answer}}{3}, 1\right)$$

Then, it is averaged over $\binom{10}{9}$ sets of human annotators. If the answers were freestyle phrases or sentences, not just one to three words, then BLEU, METEOR, or CIDEr, introduced earlier in this chapter, can be used. Gao and others in their paper *Are You Talking to a Machine? Dataset and Methods for Multilingual Image Question Answering* proposed the *Visual Turing Test* for freestyle answers. Essentially, a human judge is presented with answers from the human annotators or a machine learning model, then he or she needs to decide whether the answer comes from a human or a machine.

A popular paradigm for solving the visual question answering problem is to pose the problem as a classification problem, then use a **Convolutional Neural Network (CNN)** for encoding the image, a **recurrent neural network (RNN)** for encoding the questions, combining them, and feeding to a multilayer perceptron classifier. The following figure illustrates this concept:



The image embedding can often be done using a CNN model that is pre-trained. For example, we can use the last hidden layer of the VGG network (http://www.robots.ox.ac.uk/~vgg/research/very_deep/) as the image embedding. The pre-trained weights can be found at <http://www.cs.toronto.edu/~frossard/post/vgg16/>. Another approach is to bring question-guided attention to the image embedding.

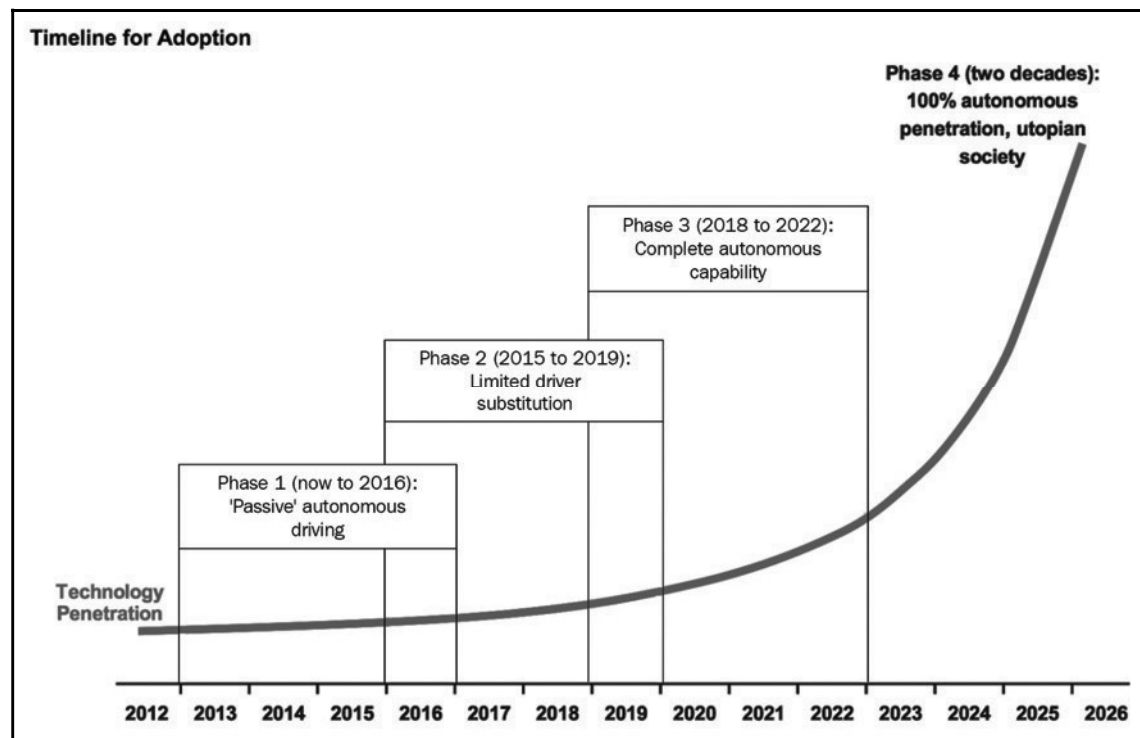
For example, we can pre-train a CNN model to produce $M \times K$ features, where M is the number of locations in the image, and K is the feature vector dimension. Then, we concatenate the question, embedding with each of the feature vectors at different locations in the image, and compute a weight. The reader can refer to <https://arxiv.org/abs/1708.02711> for details on weight computation. The final image embedding is the weighted average of the M feature vectors.

To embed the question, we cap the question to a certain length, then encode each word with a 300-dimensional vector, using a fully-connected layer and tanh non-linearity, then feed each of the word embeddings in the question to the RNN, for example, long short term memory networks. We use the final state of this RNN as the question embedding.

Finally, we point-wise multiply the image and word embeddings and feed it to the multi-layer perceptron classifier to produce the result. As of December 2017, the best performing model on the VQA dataset has 69% accuracy over all types of questions, and human accuracy is about 83%. This big gap indicates that visual question answering is a challenging AI task and there is still significant room to improve.

Multi-source based self-driving

Is autonomous driving far from us? Perhaps not. If you look at the following road map published by Morgan Stanley Research, it is expected that in around 10-15 years we can safely let cars drive for us without much interference:



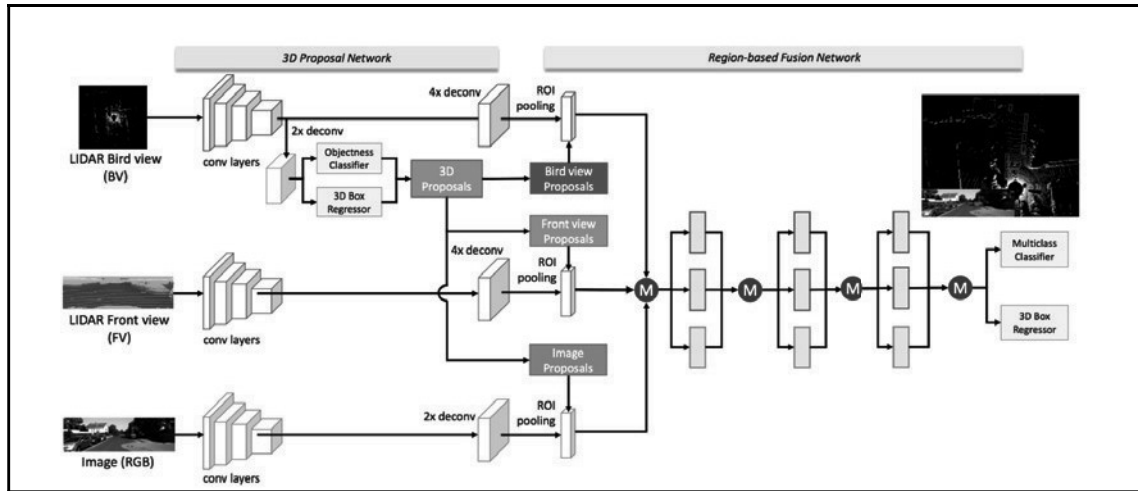
Great applications need powerful tools to realize them, and deep learning has been recognized as one such tool. Deep learning can potentially help in solving many problems that exist in the self-driving car system, for example:

- Lane detection
- Pedestrian detection
- Road sign recognition
- Traffic light detection
- Face detection/recognition
- Car detection
- Obstacle detection
- Environment recognition
- Human action recognition
- Blind-spot monitoring

And for all of these problems, it's possible that we can leverage multiple data sources to gain better accuracy and speed. Basically, a self-driving car can be equipped with various sensors, such as visible-light or infrared cameras, LIDARs, radars, ultrasonics, GPS/IMU, audio, and so on, to be able to see their environment. Data from these different modalities can be jointly utilized to make decisions and generate commands for driving. Different sources of data capture parts of the information from the environment in a certain format, which then can be combined to deal with:

- Localization and mapping: where is the car?
- Orientation—which direction is the car moving towards?
- Detection—what is surrounding the car, static and dynamic?
- Perception/scene understanding—what are detected objects, traffic lights, lanes, or moving pedestrian/cars?
- Prediction—how do detected objects change in the future?
- Motion planning—best car moving path and speed? How to move from A to B?
- Vehicle command—speed up, braking/slowing down, steering (left or right), changing lanes
- Driver state—what's the driver up to?

In a recently published work by Chen and others from Baidu, *Multi-View 3D Object Detection Network for Autonomous Driving*, the authors employ both LIDAR and cameras to acquire data, then proposed **Multi-View three-dimensional (MV3D)** networks to combine LIDAR and camera images in a complex neural network pipeline, as shown in the following figure:



The process of execution is as follows:

1. This **Multi-View 3D** networks (**MV3D**) first utilize the bird's-eye view from the LIDAR data to generate three-dimensional bounding box proposals
2. The generated bounding box is then projected to a different direction to get the two-dimensional region of interest proposals, which are then used in RGB images and LIDAR bird's-eye view projection data and front-view projection data
3. The resulting regions of interest are fed into a multilayer network to obtain the region-based fusion results

Interested readers can refer to the above mentioned paper from Chen and others for more detailed reading.

There are not many multimodality publicly-available datasets for self-driving. A well-known one is the KITTI dataset (<http://www.cvlibs.net/datasets/kitti/>). The KITTI dataset was collected by an autonomous driving platform called **Anniway**, a project of the Karlsruhe Institute of Technology and the Toyota Technological Institute at Chicago. The dataset was captured by driving around the mid-size city of Karlsruhe, in rural areas and on highways. Overall, it contains six hours of traffic scenarios at 10–100 Hz using a variety of sensor modalities, such as high-resolution color and grayscale stereo cameras, a Velodyne 3D laser scanner, and a high-precision GPS/IMU inertial navigation system.

Data has been calibrated, synchronized, and timestamped, and provides the rectified and raw image sequences. It is suitable for tasks such as stereo, optical flow, visual odometry, 3D object detection, and 3D tracking. Accurate ground truth is provided by a Velodyne laser scanner and a GPS localization system. There are up to 15 cars and 30 pedestrians visible per image. Some object labels are available in the form of 3D tracklets. The KITTI Vision Benchmark Suite also comes with a few benchmarks for each task, as well as an evaluation metric.



Note that this dataset is of non-commercial license.

For readers who are interested in applying deep learning to self-driving, there are many good resources out there on the internet, such as:

- End-to-end visual based self-driving: <https://devblogs.nvidia.com/parallelforall/deep-learning-self-driving-cars/>
- A self-driving course from MIT: <https://selfdrivingcars.mit.edu/>

Summary

In this chapter, we learned what is multimodality learning and its challenges, and some specific areas and applications in multimodality learning, including image captioning, visual question answering, and self-driving car. In the next chapter, we will deep dive into another multimodality learning area, audio-visual speech recognition. We will be covering the audio and visual feature extraction methods and models, and how to integrate them to perform reliable speech recognition.